USER PRO · TIZADA PRO

Conectar otro sistema con TIZADA PRO

Guía completa para el programador: cómo se pone en marcha en el servidor, qué archivos manda y recibe el otro sistema, cómo se conecta cada JSON y cómo queda cada archivo de producción asociado a SU venta — sin que nadie toque TIZADA.

Puesta en marcha · Qué se manda · Qué se recibe · Cómo se conectan los JSON · Aviso firmado

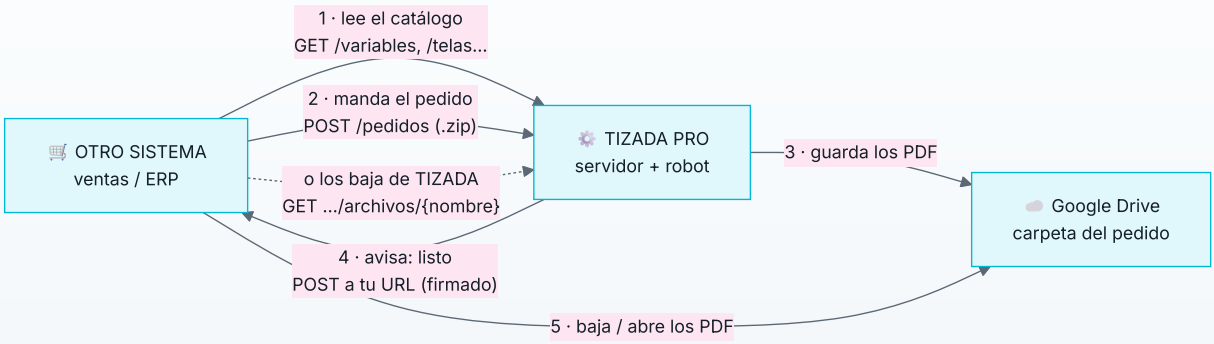
API **/api/externo/v1** · formato **tizadapro.pedido/1**

Sistema 1.0.48 · 2 de octubre de 2026

ANTES DE EMPEZAR

En una página

El otro sistema (ventas, ERP, tienda) **elige del catálogo de TIZADA** lo que se puede fabricar, **manda un paquete .zip** con el pedido y los artes, y **recibe los PDF de producción** —un archivo por mesa y la ficha técnica— con la marca de SU venta. En el medio, un **robot** de TIZADA hace todo lo que haría una persona.



Los tres actores. El otro sistema nunca entra a la pantalla de TIZADA: todo es por la API.

1 · Puesta en marcha

Una sola vez, en el servidor de TIZADA: actualizar, Node, la llave, el aviso y Google Drive. **Sección 1.**

2 · Qué viaja

Lo que manda y lo que recibe el otro sistema, y cómo se conecta cada JSON con el anterior. **Secciones 2 a 5.**

3 · Probar y usar

Código listo para usar, lista de control y problemas frecuentes. **Secciones 6 a 8.**

1 Puesta en marcha en el servidor de TIZADA

2 El recorrido de un pedido

3 Qué se manda y qué se recibe · **3b** Cómo lee TIZADA el arte · **3c** Editables (TPU / DTF / bordado)

4 Cómo se conectan los JSON

5 El pedido, el resultado y el aviso · **5b** Todos los campos de pedido.json · **5c** Moldes a medida (banderas) · **5d** Sus cuentas · **5e** Su JSON · **5f** Cí y tri ·

6 Estados y alarmas

7 Todas las rutas

8 Probarlo, lista de control y problemas frecuentes

Los archivos que acompañan esta guía

Repositorio privado **github.com/UserBreus/tizada-pro-integracion**, carpeta `integracion/`: `API.md` (la API entera), `FORMATO_PEDIDO.md`, `openapi.yaml` (para Postman), `esquemas/` (JSON Schema), `pedido_ejemplo.json`, `resultado_ejemplo.json`, `respuestas/` (la respuesta REAL de cada ruta) y `codigo/` (Python y Node listos). El mismo repositorio trae un **sistema de ventas de prueba** que hace todo esto con pantalla.

UNA SOLA VEZ · EN EL SERVIDOR DE TIZADA

1 Puesta en marcha

Lo hace quien administra el servidor de TIZADA PRO. Son siete pasos; al final, una prueba que dice si quedó bien.

1 Actualizar TIZADA PRO a una versión con la integración
La integración es del **2 de octubre de 2026** (versión 1.0.48 o posterior). Si el servidor tiene una copia anterior, primero va la actualización (el ZIP de actualización o `ACTUALIZAR-SISTEMA.bat` si la copia es de git). Se nota que está: en **Configuración** aparece la tarjeta **«Integraciones»**, y `GET /api/externo/v1/alarmas` contesta.

2 Instalar Node.js 20 o más nuevo
El **robot** que arma los pedidos que llegan es un programa de Node (`frontend/src/motor/robot/robot.mjs`) y lo arranca solo el servidor de TIZADA al iniciar. Sin Node, los pedidos se reciben pero quedan en cola. Lo que dice el robot queda en `logs/robot.log`.

3 Reiniciar TIZADA y mirar «El robot»
Configuración › Integraciones › El robot tiene que decir *«Prendido, esperando pedidos»* (punto verde). Si dice «No responde», el motivo está ahí mismo (por ejemplo, «no está instalado Node.js»).

4 Crear la llave del otro sistema
Integraciones › Llaves del otro sistema: escribir para quién es (p. ej. «Sistema de ventas») y tocar **«Crear llave»**. Se muestra **una sola vez**: «Copiar» y pasársela al programador del otro sistema por un canal seguro. TIZADA guarda sólo su huella. Para cortar el acceso: **«Anular»**.

5 El aviso al otro sistema (opcional, recomendado)
Integraciones › Aviso al otro sistema: la dirección donde el otro sistema recibe el aviso, p. ej. `https://ventas.ejemplo.com/tizada/aviso` → «Guardar». TIZADA le hace POST cuando cada pedido termina. Sin aviso, el otro sistema pregunta el estado cada pocos segundos.

6 Google Drive (para que los PDF queden en Drive)

1. En **Google Cloud** (proyecto propio): habilitar la **API de Google Drive** y crear un **ID de cliente OAuth** de tipo **«Aplicación web»**. En «URI de redireccionamiento autorizados» va la dirección de vuelta que **muestra la pantalla de Integraciones** (termina en `/api/integracion/drive/vuelta`). Bajar el **.json** del cliente.
2. En TIZADA: **«Cargar archivo de Google»** (ese .json) → **«Conectar con Google»** → aceptar con la cuenta dueña de las carpetas.
3. Pegar el enlace de la **carpeta de las tizadas** y el de la **carpeta de las fichas técnicas** → **«Guardar las carpetas»** → **«Probar»** (escribe un archivo de prueba en las dos).

Sin Drive igual funciona: los PDF quedan en el servidor y el otro sistema los baja por la API (`descarga`).

7 Que el otro sistema llegue a TIZADA (red)
El servidor del otro sistema tiene que poder llamar a `https://<servidor-de-TIZADA>/api/externo/v1` (puerto abierto / proxy). Y si hay aviso, TIZADA tiene que poder llamar a la dirección del paso 5. En internet, siempre **HTTPS**: un proxy adelante (IIS, nginx) con el certificado, como explica `DESPLIEGUE.md` §8. Si TIZADA vive bajo un subcamino (`https://…/Tizadapro`), la API queda en `https://…/Tizadapro/api/externo/v1`.

> ⚠ **Google: la app en «Prueba» vence a los 7 días**
> Mientras la app de Google Cloud esté en modo «Prueba» con usuarios externos, el permiso de Drive se corta a los **7 días** y hay que volver a tocar «Conectar con Google». Para que no venza: **publicar la app** en Google Cloud (pide una página principal y una política de privacidad públicas) o usar una cuenta de **Google Workspace** con la app «Interna».

> ✓ **La prueba de que quedó bien**
> Desde la PC del otro sistema, con la llave: `python codigo/enviar_pedido.py una_carpeta_con_pedido --solo-revisar` → tiene que decir **«Revisión: la aceptaría (200)»**. Sin `--solo-revisar` manda el pedido, espera y baja los PDF comprobando cada sha256.

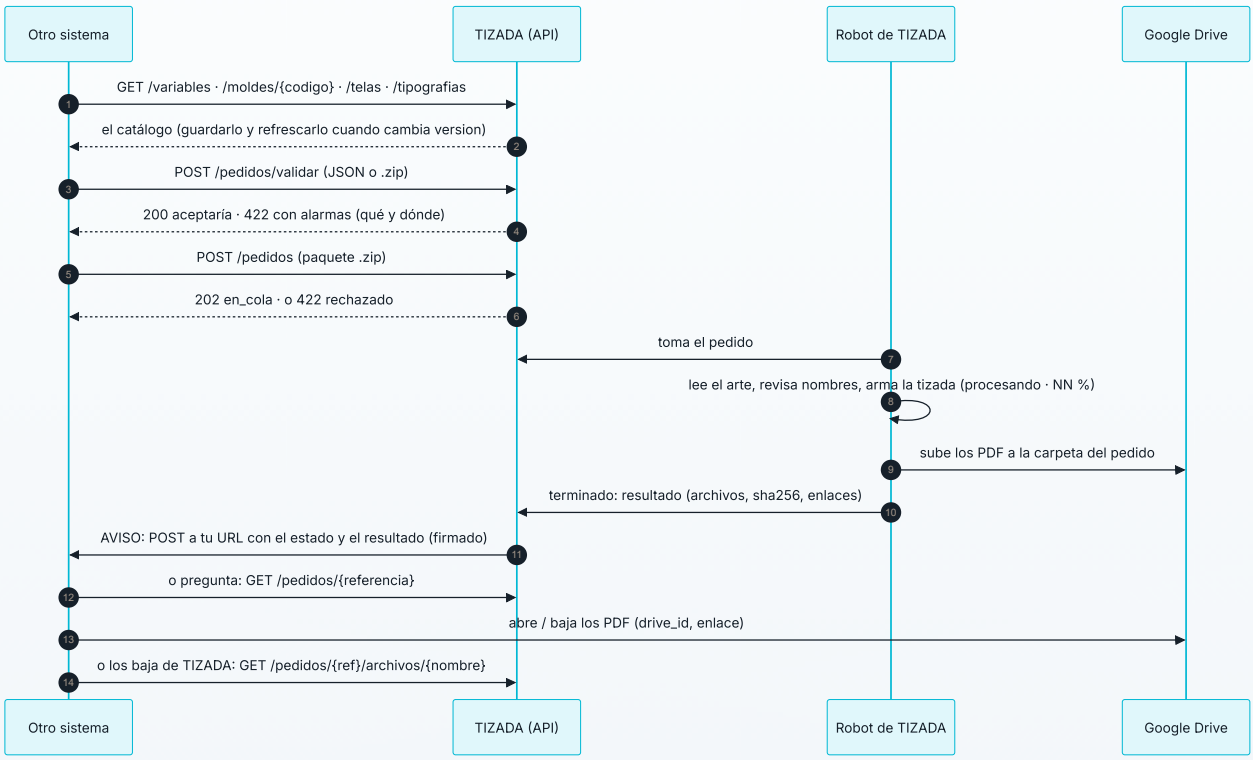
Dónde guarda TIZADA lo de la integración

```
datos/externo/
├─ config.json              llaves (sólo la huella), aviso, Drive, robot
├─ pedidos/<referencia>/    paquete.zip tal cual llegó, pedido.json, estado.json, resultado.json
├─ salida/<referencia>/     copia de los PDF (siempre, haya Drive o no)
└─ rechazos.json            lo que se rechazó al llegar (se ve en la pantalla)
```

A los **30 días** de terminado, TIZADA borra el paquete y la **copia de los PDF del servidor** (quedan el estado y el resultado como historial; Drive no se toca). Desde ahí `descarga` contesta 404: el otro sistema baja los PDF apenas termina el pedido, o usa Drive.

DE LA VENTA A LOS PDF

2 El recorrido de un pedido



Lo que hace el otro sistema está a la izquierda; todo lo demás lo hace TIZADA sola.

en_cola
esperando al robot

→

procesando
«etapa» dice en qué va (y el %)

→

listo
trae el resultado

rechazado · error · cancelado
con las alarmas

LOS ARCHIVOS

3 Qué se manda y qué se recibe

Dirección	Qué	Formato	Ruta
→ **TIZADA**	El **pedido**: `pedido.json` + los **artes** + las **tipografías** que falten	un **.zip**	`POST /pedidos`
→ **TIZADA**	Revisar el pedido antes de mandarlo	el .zip o sólo el JSON	`POST /pedidos/validar`
← **TIZADA**	El **catálogo**: variables, prendas, talles, columnas de la planilla, telas, tipografías, alarmas	JSON	`GET /variables` y otras
← **TIZADA**	La **silueta** de cada variable (para mostrarla)	JSON con contornos SVG	`GET /variables/{clave}/foto`
← **TIZADA**	El **estado** y el **resultado**	JSON	`GET /pedidos/{ref}`
← **TIZADA**	El **aviso** cuando termina	JSON firmado (POST a tu URL)	webhook
← **TIZADA**	Los **PDF**: un archivo por mesa («Mesa 1 - Bandera»...) + la ficha técnica A4	PDF	`GET …/archivos/{nombre}` o Drive

El paquete que se manda

```
OV-2026-00123.zip
├─ pedido.json                    los datos (sección 5) — en la raíz
├─ artes/jugador_camiseta.ai      .ai (con «compatible con PDF») o .pdf
├─ artes/golero_camiseta.ai
└─ tipografias/Anton-Regular.ttf  sólo las que TIZADA no tiene (.ttf / .otf)
```

Las rutas de adentro del zip son las que nombra `pedido.json`. Hasta **600 MB** y **400 archivos**. Se manda como archivo de formulario `paquete` (multipart) o como cuerpo `application/zip`.

Cómo tiene que venir el arte

El archivo

Una **mesa de trabajo por pieza**, armado sobre la base que da TIZADA para esa prenda. El dibujo **sin texto vivo** (convertido a curvas).

Las capas

`diseño` (el dibujo) · `guias` (el nombre de la pieza de cada mesa) · `Texto` (o `Nombre`) y `Número` (lo que cambia en cada prenda) · una capa `Editable <objeto>` por cada objeto que puede ir aparte (TPU, DTF, bordado).

Lo que sale

Por cada pedido: **un PDF por mesa** — `OV-2026-00123__Mesa 1 - Bandera (1,60).pdf`, una página lista para el RIP, como «Descargar todo» de TIZADA — y la **ficha técnica** `OV-2026-00123__FICHA_TECNICA.pdf` (tabla de talles y qué aplicar dónde). En Drive: las tizadas y `OV-2026-00123__resultado.json` en la carpeta del pedido, la ficha en la carpeta de fichas.

EL ARTE, CAPA POR CAPA

3b Cómo lee TIZADA el arte y los editables

TIZADA no mira colores ni adivina: **lee los nombres** de las capas y de las mesas.

Capa del arte	Para qué	Reglas
`diseño`	El dibujo de cada pieza	Sin texto vivo: todo convertido a curvas (si no, `arte-texto-vivo` y se rechaza)
`guias`	El **nombre de la pieza** de cada mesa	El nombre genérico de la pieza del molde (`GET /moldes/{molde}` → `piezas`). No importan mayúsculas, tildes, guiones ni el número del final («ESPALDA 2» = «Espalda»). **Una mesa con el nombre de su pieza se toma tal cual, sea del color que sea** (lisa, negra, rosada…).
`guias` con `#`	Arte distinto según el talle	Anteponer el talle o un rango: `#M Frente` , `#4XL-6XL Frente` . Manda el talle exacto, después el rango, después la mesa sin `#` . Si un talle queda sin mesa: `arte-variante-sin-cubrir` (aviso)
`Texto` (o `Nombre` , `jugador` , `apellido`)	El texto que cambia en cada prenda	Se llena con la columna de rol `nombre` de la planilla. Se escribe con la tipografía del arte: si TIZADA no la tiene (`GET /tipografias`), mandarla en `tipografias/` o forzar otra con `tipografia_por_campo`
`Número` (o `num` , `nro` , `00`)	El número de cada prenda	Igual que Texto, con la columna de rol `numero` . Si la planilla trae texto/número y el arte no tiene la capa: `arte-campo-sin-capa` (aviso, no se estampa)

Objetos editables: cómo sabe TIZADA que algo es TPU, DTF o bordado

1 · En el arte

Cada objeto (escudo, logo, sponsor…) en **su propia capa**, con la palabra **Editable** en el nombre: `Editable logo` . Da igual dónde va la palabra o las mayúsculas («logo editable», «EDITABLE-logo»): TIZADA la saca y lo que queda es el **nombre del objeto** → `logo` .

2 · En pedido.json

En **ese molde** (`disenos[].moldes[]`), decir qué lleva cada objeto: `"editables": { "logo": "tpu" }` Procesos: `sublimado` · `tpu` · `dtf` · `bordado` . La clave va como el objeto (`"logo"`) o como la capa (`"Editable logo"`); mayúsculas no importan. Con varios moldes, se declara en cada uno.

3 · Lo que hace

`sublimado` : se imprime con el diseño. `tpu` / `dtf` / `bordado` : **no se imprime**; va una **cruz de posición** y la ficha técnica dice qué aplicar y dónde. Sin cruz: `{"proceso": "bordado", "cruz": false}` . El resultado lo devuelve en `no_sublimado` .

Si pasa esto…	TIZADA hace	Alarma
El arte tiene la capa y el pedido no la nombra	Sale **sublimado** (impreso) y avisa	`editable-sin-declarar` (no frena)
El pedido nombra un objeto que el arte no tiene	Rechaza y dice qué objetos sí tiene el arte	`editable-desconocido`
El proceso no es uno de los cuatro	Rechaza al recibir	`editable-proceso-invalido`

TPU · DTF · BORDADO

3c Los editables al detalle: cómo llegan, cómo se detectan, qué es fijo

A · Cómo tiene que llegar en el arte

Regla	Por qué
Cada objeto en una **capa de primer nivel** del panel Capas de Illustrator (no una subcapa ni un grupo)	Al guardar, sólo las capas de primer nivel viajan con su nombre dentro del PDF. Las subcapas se pierden
Una capa por objeto: `Editable escudo`, `Editable sponsor`… (no los dos en la misma)	El proceso se elige por capa: todo lo que está en una capa lleva el mismo proceso
El nombre de la capa **contiene la palabra «editable»**	Es lo único que mira TIZADA para saber que es editable
Adentro: el objeto tal cual (vectores, una imagen PNG/JPG incrustada o texto), **en su lugar real sobre la pieza**	Ahí mismo va la cruz. Puede estar en una o en varias mesas: en cada mesa donde haya algo de esa capa, el objeto está en esa pieza
El objeto **no** se repite en la capa `diseño`	Lo que está en `diseño` se imprime siempre: el TPU saldría impreso igual
.ai guardado con «Crear archivo compatible con PDF» (o .pdf con capas)	Sin eso el archivo no trae las capas y no se ve ningún editable

B · Cómo lo detecta TIZADA, paso a paso

1. Abre el arte y lee la lista de capas del PDF.
2. Una capa es editable si su nombre tiene **«editable»** en cualquier parte, con mayúsculas o minúsculas.
3. El **nombre del objeto** es el nombre de la capa sin esa palabra ni los espacios, guiones, guiones bajos o puntos que la rodean.
4. En cada mesa junta todo lo que hay en esa capa y mide dónde está y qué tamaño tiene.
5. Busca en `editables` del pedido (en ese molde) una clave con ese nombre (sin importar mayúsculas) y aplica el proceso.

Capa en el arte	Objeto (clave en el pedido)
`Editable logo`	`logo`
`Logo editable`	`Logo`
`EDITABLE_Escudo` · `Editable-Escudo`	`Escudo` (el mismo)
`Editables sponsor`	`sponsor`
`Editable TPU`	`TPU` ⚠ es sólo el nombre, ver abajo
`Editable` (sola)	`Editable`

C · Lo que está fijo en TIZADA (no se configura)

Qué	Valor fijo
La palabra que marca un editable	`editable` (o `editables`), en cualquier parte del nombre de la capa
Los procesos (valor en `editables`)	**Sólo cuatro:** `"sublimado"` · `"tpu"` · `"dtf"` · `"bordado"`. Se aceptan con mayúsculas (`"TPU"`). Cualquier otra palabra («vinilo», «serigrafía», «estampado») → `editable-proceso-invalido`
Si el objeto no se nombra en el pedido	Sale `sublimado` (impreso) + aviso `editable-sin-declarar`
La marca en la tela (TPU, DTF, bordado)	Cruz de **3 cm** de punta a punta, trazo de 1,6 mm, **negra con borde blanco**, centrada donde estaba el objeto, con la letra del proceso: **T** = TPU · **D** = DTF · **B** = Bordado
Sin marca	`{"proceso": "tpu", "cruz": false}`: no se imprime y no queda nada en la tela (sólo en la ficha técnica)
Dónde se declara	En cada molde: `disenos[].moldes[].editables`

> ⚠ **El nombre de la capa NO elige el proceso**
>
> Una capa «Editable TPU» crea un objeto que se **llama** «TPU», pero si el pedido no dice `"editables": {"TPU": "tpu"}` sale **sublimado**. El proceso lo decide siempre el pedido.

D · Ejemplo completo

El panel Capas del .ai (capas de primer nivel):

```
camiseta_jugador.ai
├─ guias              «Frente», «Espalda», «Manga»…
├─ Texto              el apellido
├─ Número             el número
├─ Editable escudo    el escudo, en el Frente
├─ Editable sponsor   el sponsor, en el Frente
├─ Editable logo      el logo de la marca, en la Manga
└─ diseño             todo lo demás (sin esos 3 objetos)
```

En pedido.json, dentro de ese molde:

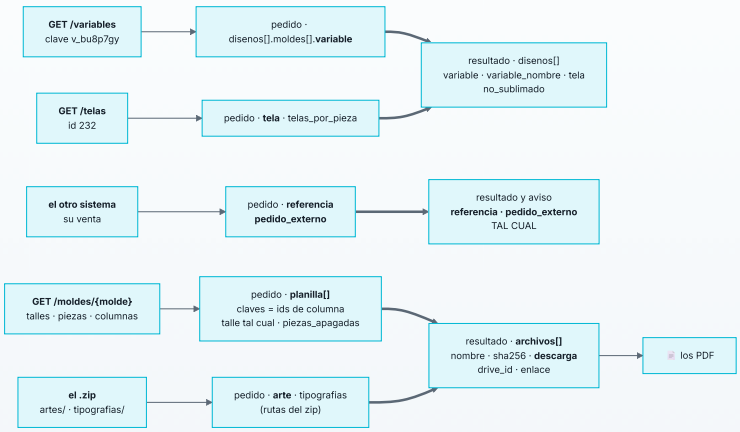
```
"editables": {
  "escudo":  "bordado",
  "sponsor": "sublimado",
  "logo":    { "proceso": "tpu", "cruz": false }
}
```

Objeto	En la tizada (PDF de la mesa)	En la ficha técnica	En el resultado
`escudo`	No se imprime: cruz con **B** en el Frente	Bordado, en el Frente	`"no_sublimado": {"escudo": "bordado", "logo": "tpu"}`
`sponsor`	Se imprime con el diseño	—	
`logo`	No se imprime y **no queda marca**	TPU, en la Manga	

EL MAPA DE LOS DATOS

4 Cómo se conectan los JSON

Cada valor del pedido sale de algo que publica TIZADA, y cada cosa del resultado vuelve atada a lo que mandó el otro sistema. Ésta es la cadena:



Flechas finas: «de dónde sale el valor». Flechas gruesas: «vuelve atado a».

En el pedido	Sale de	Cómo va
`disenos[].moldes[].variable`	`GET /variables` → `clave`	La clave `v_…` . **Una variable por prenda** en cada diseño
`…tela` , `…telas_por_pieza` (valores)	`GET /telas` → `id` (y las que admite la prenda: `telas` del molde)	El **id**, como texto: `"232"`
`telas_por_pieza` (claves), `piezas_apagadas`	`GET /moldes/{molde}` → `piezas`	Nombre **genérico**: «Cuello», no «Cuello 2»
Claves de cada fila de `planilla[]`	`planilla.columnas[].id` del molde	`talle` , `nombre` , `numero` , `manga` , `cantidad` …
Valor de la columna de talle	`talles[]` del molde (y `planilla.columna_talle`)	**Tal cual**: `"M"` , `"2XL"` , `"10"`
Valores de columnas con lista (p. ej. manga)	`columnas[].opciones` y `opciones_de_pieza`	Una opción, o varias: `"Corta + Larga"`
`planilla[].diseno`	el `nombre` de uno de los `disenos[]` del mismo pedido	Obligatoria si hay más de un diseño
`arte` , `tipografias`	las rutas de adentro del **.zip**	`"artes/jugador.ai"`
`editables` (claves)	las capas `Editable X` del arte	`X` → `sublimado` · `tpu` · `dtf` · `bordado`
`referencia` , `pedido_externo`	el otro sistema	**Vuelven tal cual** en el resultado y el aviso

EJEMPLOS REALES

5 El pedido, el resultado y el aviso

pedido.json

```jsonc
{
  "formato": "tizadapro.pedido/1",
  "referencia": "OV-2026-00123",          ← única · carpeta en Drive y prefijo de cada PDF
  "pedido_externo": { "id": 48211, "sucursal": "Montevideo" },  ← libre: vuelve TAL CUAL
  "cliente": "Club Atlético Ejemplo",
  "opciones": { "si_piezas_en_blanco": "rechazar", "carpeta": "2026-10/OV-2026-00123" },
  "mesas": { "modo": "normal" },               ← o "una_por_fila", o "por_talles" + grupos
  "disenos": [
    { "nombre": "JUGADOR",
      "moldes": [ {                              ← cada entrada = una VARIABLE elegida
        "variable": "v_bu8p7gy",                 ← GET /variables (Cuello redondo)
        "arte": "artes/jugador_camiseta.ai",     ← ruta dentro del .zip
        "tela": "232", "telas_por_pieza": { "Cuello": "486" },   ← ids de GET /telas
        "editables": { "escudo": "sublimado", "logo": "tpu" } } ] },
    { "nombre": "GOLERO", "arte": "artes/golero_camiseta.ai",
      "moldes": [ { "variable": "v_t1mhcmu", "tela": "492",
        "editables": { "escudo": { "proceso": "bordado", "cruz": true } } } ] } ],
  "planilla": [                                  ← una fila por prenda · claves = ids de columna
    { "diseno": "JUGADOR", "talle": "M", "nombre": "PÉREZ", "numero": "10", "manga": "Corta", "cantidad": 1 },
    { "diseno": "GOLERO",  "talle": "L", "nombre": "GÓMEZ", "numero": "1",  "manga": "Larga" } ]
}
```

Lo que vuelve: GET /pedidos/{referencia} (y el cuerpo del aviso)

```jsonc
{ "referencia": "OV-2026-00123", "estado": "listo", "etapa": "terminado",
  "pedido_externo": { "id": 48211, "sucursal": "Montevideo" },       ← el de la venta, tal cual
  "resultado": {
    "formato": "tizadapro.resultado/1", "tizada_id": 912,
    "destino": { "tipo": "drive", "carpeta_id": "1mJl5…", "carpeta_enlace": "https://drive.google.com/drive/folders/…" },
    "archivos": [
      { "tipo": "tizada", "nombre": "OV-2026-00123__HOJA_g0_Jacquard_Charr_a__1_83_.pdf",
        "bytes": 1127490, "sha256": "9c1e…",                  ← comprobar al bajarlo
        "descarga": "/api/externo/v1/pedidos/OV-2026-00123/archivos/…pdf",
        "drive_id": "1AbC…", "enlace": "https://drive.google.com/file/d/…/view",
        "tela": "Jacquard Charrúa (1,83)", "mesas": 1, "consumo_cm": 321, "aprovechamiento": 74.6 },
      { "tipo": "ficha", "nombre": "OV-2026-00123__FICHA_TECNICA.pdf", … } ],
    "disenos": [ { "nombre": "GOLERO", "moldes": [ { "variable": "v_t1mhcmu", "variable_nombre": "cuello V",
        "no_sublimado": { "escudo": "bordado" } } ] } ],
    "alarmas": [ avisos que quedaron (no frenan) ] } }
```

El aviso y su firma

Lo que llega

`POST` a la dirección del aviso, con el mismo JSON de arriba y los encabezados `X-Tizada-Referencia` y `X-Tizada-Firma`. Se manda cuando el pedido queda **listo**, **rechazado** o **error**. Contestar **2xx**; si no, reintenta a los 5 s, 30 s y 2 min.

Cómo se comprueba

`firma = HMAC-SHA256(clave = sha256_hex(llave), mensaje = cuerpo crudo)` en hexadecimal. Si no coincide, **ignorarlo**. Código listo: `codigo/recibir_aviso.py` y `firmaValida()` en `codigo/cliente.mjs`.

SIN QUE FALTE NINGUNO

5b Todos los campos de pedido.json

El ejemplo de arriba usa los más comunes. Ésta es la lista **completa** de lo que TIZADA lee; cualquier otro campo se ignora.

Cabecera

Campo		Qué es
`formato`	obligatorio	Siempre `"tizadapro.pedido/1"`
`referencia`	obligatorio	El número del pedido en el otro sistema. Único. Letras, números, punto, guion y guion bajo; hasta 64
`pedido_externo`	opcional	Objeto libre (id, sucursal, fecha…). Vuelve tal cual
`cliente`	opcional	Texto, para reconocerlo en la pantalla de TIZADA
`aviso_url`	opcional	Dónde avisar al terminar este pedido. Sólo vale si es del mismo sitio que la dirección cargada en Integraciones (si no: `aviso-url-no-permitida`)
`reemplazar`	opcional	`true` para rehacer un pedido que ya salió con la misma referencia
`opciones` · `mesas`	opcional	Ver las tablas de abajo
`disenos[]` · `planilla[]`	obligatorio	Los diseños con sus moldes, y una fila por prenda

Cada diseño (`disenos[]`) y cada molde (`disenos[].moldes[]`)

Campo	Dónde	Qué es
`nombre`	diseño · obligatorio	JUGADOR, GOLERO… Único. Es lo que va en `planilla[].diseno`
`arte`	diseño o molde	Ruta del arte dentro del .zip. El del diseño vale para sus moldes que no traigan el suyo
`tipografias`	diseño o molde · opcional	Rutas de las .ttf/.otf que usa el texto/número y TIZADA no tiene (`GET /tipografias`)
`tipografia_por_campo`	diseño · opcional	Forzar una tipografía del catálogo para un campo: `{"numero": "Anton Regular", "nombre": "Bebas Neue"}`
`moldes[]`	diseño · obligatorio	Una entrada por prenda (una variable por prenda)
`variable`	molde · obligatorio	La clave `v_…` de `GET /variables` (o su nombre, si no se repite)
`molde`	molde · opcional	Código `prod_…` . Sólo si la variable va por nombre y ese nombre está en varias prendas
`tela` · `telas_por_pieza`	molde	Id de tela principal (obligatorio) y excepciones por pieza genérica: `{"Cuello": "486"}`
`piezas_apagadas`	molde · opcional	Piezas que **no** se hacen en este pedido: `["Tapa costura"]`
`editables`	molde · opcional	Qué lleva cada objeto editable del arte (TPU, DTF, bordado…). **Ver sección 3b**
`medida`	molde · sólo prendas a medida	`{"ancho_m": 1.5, "alto_m": 0.9}` = la medida del **diseño** en metros (0,05 a 50); TIZADA le suma el margen. Lo indica `a_medida` en `GET /variables` ; esa fila de la planilla no lleva talle. En el círculo, `{"diametro_m": 1}` . **Ver 5c y 5f**
`terminacion`	molde · a medida · opcional	`dobladillo` , `overlock` o `sin` (sin margen); si no viene, la del molde. **Ver 5g**
`forma` · `orientacion`	molde · a medida · opcional	`rectangulo` (si no viene), `circulo` o `triangulo` ; en el triángulo, la punta: `abajo` (si no viene), `arriba` , `izq` , `der` . **Ver 5f**
`tiras`	molde · a medida · opcional	Marcas para coser las tiras: `{"lleva": true, "lados": {"arriba": 5, "abajo": 5, "izq": 0, "der": 0}}` ; cada lado cuenta las 2 puntas. Sin esto, las del molde. **Ver 5e**

`opciones`

Campo	Valores	Si no viene
`si_falta_tipografia`	`"rechazar"` · `"predeterminada"` (usa la predeterminada y avisa)	rechazar
`si_texto_no_entra`	`"achicar"` (avisa) · `"rechazar"`	achicar
`si_piezas_en_blanco`	`"rechazar"` · `"seguir"` (salen sin diseño y avisa)	rechazar
`perfil_color`	Nombre del perfil .icc a forzar	el del sistema
`carpeta`	Subcarpeta de Drive: `"2026-10/OV-2026-00123"`	la referencia

`mesas`

modo	Qué hace
`"normal"`	Lo de siempre (por defecto)
`"una_por_fila"`	Una mesa por fila; la cantidad pasa a ser copias. Necesita columna de cantidad
`"por_talles"`	Los talles de cada grupo comparten mesa: `"grupos": [["S","M"],["L","XL"]]`

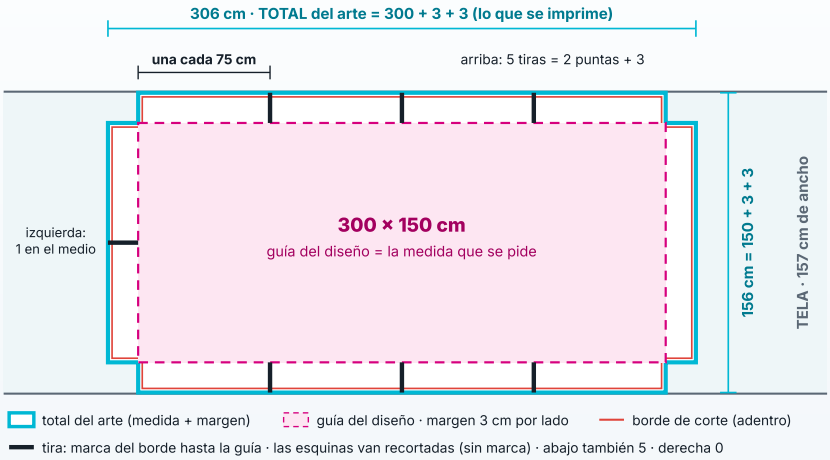
`planilla[]`

Claves = ids de columna del molde. Todo como texto (`"07"` con su cero). Talle tal cual el molde; opciones de la lista (`"Corta + Larga"` para varias); `cantidad` entero ≥ 1 (vacía = 1); `diseno` obligatoria si hay más de un diseño.

BANDERAS Y OTROS RECTÁNGULOS

5c Moldes a medida: qué es y qué hay que preguntar

Un **molde a medida** es una prenda de **una sola pieza rectangular** (una bandera, un cartel) que **no tiene talles**: el tamaño lo pone cada pedido. Se reconoce porque en `GET /variables` y en `GET /moldes/{codigo}` trae el bloque `a_medida`. El otro sistema le pregunta a la persona **tres cosas** y TIZADA hace el resto.



Bandera de 3,00 × 1,50 con 3 cm de margen en la tela «Bandera (1,60)»: 306 × 156 entra girada (156 ≤ 156,7). El margen está dibujado más ancho para que se vea.

1 · La medida del diseño

Ancho y alto **del diseño** en metros: lo que queda a la vista, adentro del margen. Entre `medida_minima_m` y `medida_maxima_m`. TIZADA le suma el margen: 3,00 × 1,50 con 3 cm por lado sale de **3,06 × 1,56**.

2 · La tela

Una de `a_medida.telas`. Entra si el lado del **total** que va atravesado ≤ `entra_hasta_cm` (si `gira`, sirve el más corto) y el otro ≤ `largo_maximo_cm`. Si no: `tela-no-entra`, con cuánto se pasa y lo máximo.

3 · Las tiras (opcional)

Si lleva y cuántas marcas en **cada lado** (arriba, abajo, izq, der). El número **cuenta las 2 puntas**: 5 = 2 esquinas + 3 en el medio; 1 = una en el medio; 0 = ese lado sin tiras. Si no se manda: las del molde.

Lo que hace TIZADA con eso

Arma la pieza del **total** (medida + margen), la acomoda en la tela (girada si así entra), imprime el diseño hasta el borde, el **borde de corte** por dentro y las **marcas de las tiras** del color y grosor del molde; en la **ficha técnica** agrega «MEDIDA Y TERMINACIÓN» con todas estas medidas, el **plano de la pieza** acotado, el borde en corte **paso a paso** (cómo se dobla y dónde se cose), la esquina o punta **cortada y doblada** y los pasos para terminarla.

MOLDE A MEDIDA · ANTES DE MANDAR

5d Las cuentas, y la ruta que las hace por vos

El otro sistema puede hacer estas cuentas solo, con lo que publica `a_medida`, para mostrarle a la persona **en el momento** si su bandera entra y hasta cuánto puede pedir. O se las puede preguntar a TIZADA: dan exactamente lo mismo.

Las cuentas

Qué	Cómo se calcula	En el ejemplo
Total del arte	ancho = ancho del diseño + margen izq + der · alto = alto del diseño + margen arriba + abajo (el borde de corte va **adentro**)	300 + 3 + 3 = **306** · 150 + 3 + 3 = **156** cm
¿Entra en la tela?	lado atravesado ≤ `entra_hasta_cm` y el otro ≤ `largo_maximo_cm`; si `gira`, probar también al revés	156 ≤ 156,7 ✓ (girada)
Lo máximo	`entra_hasta_cm` − el margen de esos dos lados, redondeado al cm para abajo	156,7 − 6 → **1,50 m** de alto
Tiras de un lado	N contando las 2 esquinas, a distancias iguales sobre la **guía** → una cada **largo de la guía ÷ (N − 1)**; con 1, en el medio. Con dobladillo la esquina va **recortada** y no lleva marca (cuenta igual); con overlock lleva una, en diagonal	arriba 5: 300 ÷ 4 = **75 cm** · marca de 3 cm
Total de marcas	las del medio de cada lado (+ las esquinas con overlock: una sola aunque la pidan 2 lados)	3 + 3 + 1 = **7**

`POST /a_medida/calcular` — TIZADA hace las cuentas

```
// lo que se manda (no guarda nada)
{ "variable": "v_cb7fd79", "ancho_m": 3, "alto_m": 1.5,
  "tela": "44",                          ← opcional: si no viene, todas las del molde
  "tiras": { "arriba": 5, "abajo": 5, "izq": 1 } }   ← opcional: si no viene, las del molde
```

```
// lo que contesta: 200 (o 422 con las alarmas si algo no sirve)
{ "medida_diseno_cm": { "ancho": 300, "alto": 150 },
  "total_cm": { "ancho": 306, "alto": 156 },
  "margen_cm": { … }, "borde_corte_mm": 2, "gira": true, "largo_maximo_cm": 2999.5,
  "telas": [ { "id": "44", "nombre": "Bandera (1,60)", "entra_hasta_cm": 156.7,
               "entra": true, "motivo": null } ],     ← si no entra: motivo = cuánto se pasa y lo máximo
  "entra_en_alguna": true,
  "tiras": { "lleva": true, "total_marcas": 7, "grosor_mm": 1, "color": "negra con contorno blanco",
    "lados": { "arriba": 5, "abajo": 5, "izq": 1, "der": 0 },
    "por_lado": [ { "lado": "arriba", "marcas": 5, "cada_cm": 75, "en_el_medio": false,
                    "largo_del_lado_cm": 300, "largo_marca_cm": 3 },
                  { "lado": "izq", "marcas": 1, "cada_cm": null, "en_el_medio": true, … }, … ] },
  "ficha": [ { "etiqueta": "Total del arte (con el margen)", "valor": "306 × 156 cm" }, … ],
  "alarmas": [] }
```

El molde va por `variable` (o `molde`). Si no es a medida: **422**; si no existe: **404**. Pide la llave como todo `/api/externo/v1`.

EL JSON DE UN MOLDE A MEDIDA

5e Qué publica TIZADA y cómo se manda el pedido

1 · leer `GET /variables` → bloque `a_medida` → **2 · preguntar** medida del diseño, tela y tiras → **3 · calcular** `POST /a_medida/calcular` (opcional) → **4 · mandar** `POST /pedidos` con `medida` y `tiras`

1 · Lo que publica TIZADA

```
"a_medida": {                          ← en GET /variables y en GET /moldes/{codigo}
  "pieza": "Bandera",
  "medida_minima_m": 0.05, "medida_maxima_m": 50,        ← la medida del DISEÑO, en metros
  "medida_de_muestra_m": { "ancho_m": 1, "alto_m": 1 },
  "margen_cm": { "arriba": 3, "abajo": 3, "izq": 3, "der": 3 },   ← total = medida + margen
  "borde_corte_mm": 2,                                   ← va ADENTRO del total
  "gira": true, "largo_maximo_cm": 2999.5,
  "telas": [ { "id": "44", "nombre": "Bandera (1,60)", "ancho_mesa_cm": 157, "entra_hasta_cm": 156.7 } ],
  "tiras": { "lleva_por_defecto": false, "maximo_por_lado": 50,
             "lados_por_defecto": { "arriba": 0, "abajo": 0, "izq": 0, "der": 0 },
             "grosor_mm": 1, "color": "negra con contorno blanco" },   ← grosor y color: los pone TIZADA
  … y además «total», «regla_entra» y «tiras.regla»: las reglas, en una frase }
```

2 · El molde dentro de pedido.json

```
{ "variable": "v_cb7fd79",
  "arte": "artes/bandera.ai",
  "tela": "44",
  "medida": { "ancho_m": 3, "alto_m": 1.5 },
  "tiras": { "lleva": true,
    "lados": { "arriba": 5, "abajo": 5,
               "izq": 1, "der": 0 } } }
```

Va en `disenos[].moldes[]`. La fila de la `planilla` de un molde a medida **no lleva talle**.

Las formas de mandar `tiras`

Si «tiras» es…	Qué hace
(no viene)	las del molde (`lados_por_defecto`)
`false` · `true`	no lleva · lleva las del molde
`{lleva, lados}`	la forma completa; un lado que no viene es 0
`{"arriba": 5, "der": 3}`	los lados sueltos (lleva si alguno > 0)

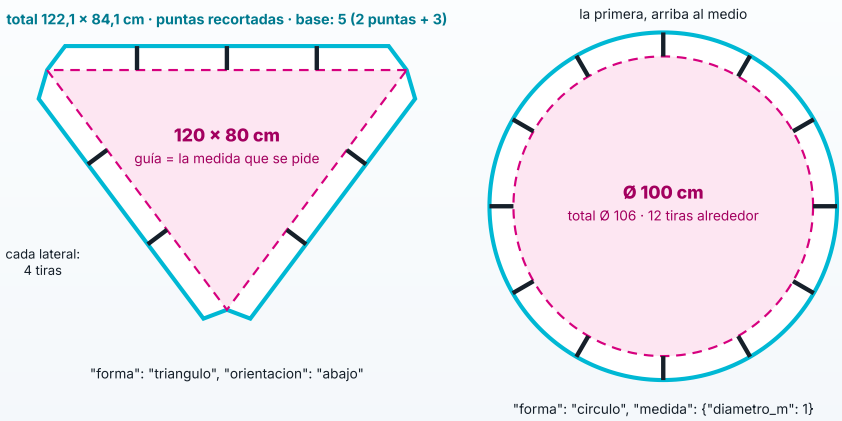
Lados: `arriba` , `abajo` , `izq` / `izquierda` , `der` / `derecha` ; enteros de 0 a 50, **contando las 2 puntas**.

Alarma	Cuándo	Qué hacer
`medida-falta` · `medida-invalida`	Falta `medida` o no son metros de 0,05 a 50	`{"ancho_m": 3, "alto_m": 1.5}` con punto
`tela-no-entra`	El total no entra en la tela; el mensaje dice cuánto se pasa y lo máximo	Otra tela o una medida menor
`tiras-invalidas`	Un lado que no existe o un número fuera de 0 a 50	Usar arriba/abajo/izq/der con enteros
`tiras-sin-margen`	Tiras en un lado sin margen (no hay guía hasta dónde llegar)	Pedirlas en lados con `margen_cm` > 0
`tiras-no-a-medida` `aviso`	Tiras en un molde que no es a medida: se ignoran	Sacarlas de ese molde

MOLDE A MEDIDA · OTRAS FORMAS

5f Círculos y triángulos: lo mismo, con dos datos más

El mismo molde a medida puede salir **rectángulo** (lo de siempre), **círculo** o **triángulo**: la forma la elige **cada pedido**. Todo lo de 5c a 5e vale igual; cambian sólo tres cosas: se manda `forma` (y en el triángulo, hacia dónde va la **punta**), el **margen va parejo** todo alrededor (el mayor de `margen_cm`) y las **tiras** se cuentan distinto.



Triángulo 1,20 × 0,80 (punta abajo) y círculo de Ø 1,00, los dos con 3 cm de margen. Celeste: el total que se imprime · rosa punteado: la guía del diseño · negro: las tiras. El margen está dibujado más ancho para que se vea.

En pedido.json

```
// triángulo: ancho y alto de su CAJA + la punta
{ "variable": "v_cb7fd79", "tela": "44",
  "arte": "artes/banderin.ai",
  "forma": "triangulo", "orientacion": "abajo",
  "medida": { "ancho_m": 1.2, "alto_m": 0.8 },
  "tiras": { "lleva": true,
    "lados": { "base": 5, "laterales": 4 } } }

// círculo: una sola medida, el diámetro
{ "variable": "v_cb7fd79", "tela": "44",
  "arte": "artes/escudo.ai",
  "forma": "circulo",
  "medida": { "diametro_m": 1 },
  "tiras": { "alrededor": 12 } }
```

`forma` y `orientacion` pueden ir en el molde o dentro de `medida`, con o sin tilde. Sin `forma`: rectángulo. `POST /a_medida/calcular` acepta lo mismo y contesta `forma`, `orientacion`, `total_cm` y las tiras por lado.

Las cuentas

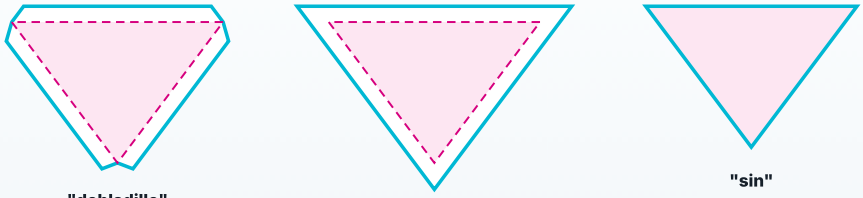
Qué	Círculo	Triángulo
Total	Ø diseño + 2 × margen: 100 → **Ø 106**	cada lado + el margen, con las **puntas recortadas** (al doblar por la guía se superponen sin sobrar): 120 × 80 → **122,1 × 84,1**
Punta	—	`abajo` (banderín) · `arriba` · `izq` · `der`
Tiras	`alrededor`: N parejas (hasta 200), la 1.ª arriba; una cada perímetro ÷ N (333 ÷ 12 = 27,75 cm)	`base` y `laterales` (cada uno) **contando las 2 puntas**: sobre la guía: base 5 → cada 30 cm · laterales 4 → cada 33,33 cm
Esquinas	—	recortadas hasta la punta de la guía: cada solapa doblada llega justo al otro lado y se **superpone** con la vecina, como la esquina del rectángulo. **La punta no lleva marca** (el corte ya la marca) pero cuenta
Borde de corte	adentro del total, como en el rectángulo	

¿Entra en la tela? La regla de 5d con la caja del total (el círculo, por su diámetro). Forma o punta que no existe: `forma-invalida`. Los lados de tiras que no son de esa forma no se usan.

MOLDE A MEDIDA · EL BORDE

5g La terminación del borde, y tiras sin marcar

Cada pedido elige cómo se termina el borde de la pieza: **dobladillo**, **overlock** o **sin margen**. Cambia cuánto se agrega alrededor del diseño y, en el triángulo, la forma de las puntas. Si no se manda, va la del molde (`terminacion_por_defecto`).



"dobladillo"
el margen del molde, doblado · puntas recortadas

"overlock"
overlock_cm alrededor · la forma tal cual

"sin"
sin margen · la pieza es el diseño

Celeste: lo que se imprime · rosa punteado: el diseño. Los márgenes están dibujados más anchos para que se vean.

En pedido.json

```
{ "variable": "v_cb7fd79", "tela": "44",
  "arte": "artes/banderin.ai",
  "forma": "triangulo",
  "terminacion": "overlock",      ← dobladillo · overlock · sin
  "medida": { "ancho_m": 1.2, "alto_m": 0.8 },
  "tiras": { "lleva": true,
             "marcar": false,     ← no se imprimen; van en la ficha
             "lados": { "base": 5, "laterales": 4 } } }
```

`terminacion` va en el molde o dentro de `medida`, con o sin tilde («sin margen» también). Si no existe: `terminacion-invalida`. `POST /a_medida/calcular` acepta lo mismo y contesta `terminacion` y `tiras.marcadas_en_tizada`.

Qué cambia con cada una

	Margen	Rectángulo y triángulo	Tiras
dobladillo	`margen_cm`	esquinas y puntas recortadas	sin marca en la esquina
overlock	`overlock_cm` parejo	esquinas y puntas enteras	una por esquina, en diagonal
sin	0	el diseño	sólo en la ficha

Tiras que no se marcan

Con `"marcar": false` las marcas **no se imprimen** en la tizada (en la pantalla de TIZADA se ven como guía), pero la **ficha técnica** trae cuántas tiras van y cada cuánto se cosen. Sin margen siempre es así (no hay guía hasta dónde marcar). Sin `marcar`: lo del molde (`tiras.marcar_por_defecto`).

Las marcas salen siempre **negras con contorno blanco** (se ven sobre cualquier diseño): el color no se elige.

La costura, en la plantilla

Con **dobladillo**, la plantilla de Illustrator y CorelDRAW (y la guía .ai) trae además una línea **punteada naranja** adentro de la guía del diseño, a la misma distancia (el margen): ahí pasa la costura. Sirve para ver, antes de subir el arte, si algo importante queda debajo.

CUANDO ALGO NO ESTÁ BIEN

6 Estados y alarmas

estado	Qué significa	Qué hace el otro sistema
`en_cola`	Recibido, esperando al robot	Esperar
`procesando`	Haciéndose; `etapa` es texto para mostrar, con `(NN %)` si se sabe el avance	Mostrar la etapa
`listo`	Terminó: viene `resultado`	Guardar los archivos (sha256, descarga, drive_id, enlace) en la venta
`rechazado`	El arte, las tipografías o el plan tienen algo que frena. No salió nada	Mostrar las alarmas, corregir y mandar de nuevo
`error`	Falla del sistema después de 3 intentos	Avisar a quien administra TIZADA
`cancelado`	Lo canceló el otro sistema (`DELETE`) o una persona	—

Cada alarma: `{ codigo, frena, etapa, mensaje, donde: { campo, fila } }`. **frena: true** rechaza; **false** es un aviso. `donde.campo` dice exactamente dónde (`disenos[1].moldes[0].tela` , `planilla[2].talle`). Son **80**, con qué significan y qué hacer: `GET /alarmas` (no pide llave).

datos
al recibir, en el acto (lo dice /validar)

→

arte
al abrir el arte (segundos después)

→

tizada
al armarla y guardar los PDF

Para no caer en...	Revisar antes contra...
`variable-desconocida` · `variable-ambigua`	`GET /variables` (mandar la clave `v_…`)
`molde-repetido`	Una variable por prenda en cada diseño
`tela-desconocida` · `tela-no-usable` · `tela-no-permitida`	`GET /telas` (`usable`) y las `telas` del molde
`medida-falta` · `medida-invalida` · `forma-invalida` · `terminacion-invalida` · `tela-no-entra` · `tiras-invalidas` · `tiras-sin-margen`	Prenda **a medida** (`a_medida` en `GET /variables`): `medida` del diseño en metros, que el total entre en la tela y las `tiras` bien escritas — o preguntar antes a `POST /a_medida/calcular` (5c a 5g)
`arte-otra-medida`	Prenda **a medida**: el arte se armó con la plantilla de **otra medida** (sus guías no coinciden con la pieza y el diseño no la cubre). Pedir la plantilla con la medida del pedido (`POST /plantillas`) y armar el arte ahí
`talle-inexistente`	`talles[]` del molde
`opcion-inexistente` · `opcion-sin-piezas`	`columnas[].opciones` · `opciones_de_pieza`
`columna-obligatoria-vacia`	`columnas[].obligatoria`
`pieza-desconocida`	`piezas[]` del molde
`referencia-ya-generada`	La misma referencia ya está lista: `"reemplazar": true` para rehacerla
`piezas-en-blanco` · `arte-pieza-sin-mesa` · `editable-desconocido` · `tipografia-falta`	El arte (etapa arte): una mesa por pieza, las capas Editable y las tipografías del paquete
`editable-proceso-invalido` · `editable-sin-declarar`	Los editables (sección 3b): proceso válido y cada capa «Editable …» nombrada
`arte-ilegible` · `arte-texto-vivo` · `arte-mesa-vacia` · `arte-campo-sin-capa`	.ai «compatible con PDF», textos en curvas, nombre de pieza en `guias` , capas Texto/Número (sección 3b)

REFERENCIA RÁPIDA

7 Todas las rutas

Base: `https://<servidor-de-TIZADA>/api/externo/v1` · encabezado `X-Api-Key: tzp_…` · todo en UTF-8 · el detalle de cada una en `API.md` y `openapi.yaml`.

Método y ruta	Para qué	Contesta
`GET /variables`	Lo que se elige: las variables de cada prenda lista	200
`GET /variables/{clave}/foto`	Silueta vectorial de sus piezas	200 · 404 (`sin_calcular`)
`GET /moldes` · `/moldes/{codigo}`	Las prendas · todo lo de una (talles, piezas, planilla, opciones, telas)	200 · 404
`GET /moldes/{codigo}/foto`	Silueta de la prenda entera	200 · 404
`GET /telas` · `/tipografias`	Telas por id · tipografías que ya tiene	200
`GET /alarmas` · `/formato`	Catálogo de alarmas · formatos, límites, ejemplos (sin llave)	200
`POST /a_medida/calcular`	Molde a medida: total, si entra en cada tela (y si no, cuánto se pasa y lo máximo), tiras y ficha. No guarda nada	200 · 422 · 404
`POST /pedidos/validar`	Revisar sin guardar (JSON o .zip)	200 · 422
`POST /pedidos`	Mandar el pedido (.zip)	202 · 422 · 415
`GET /pedidos/{ref}`	Estado y resultado	200 · 404
`GET /pedidos/{ref}/archivos/{nombre}`	Bajar un PDF (`archivos[].descarga`)	200 (PDF) · 404
`DELETE /pedidos/{ref}`	Cancelar si todavía no se generó	200 · 409 · 404
Aviso (webhook)	TIZADA → POST a tu URL, firmado	contestar 2xx

La llave

Sin llave o con una anulada: `401 {"error": "falta la llave o no es válida (encabezado X-Api-Key)"}`. La llave sólo abre `/api/externo/v1` y firma los avisos: guardarla en el **servidor** del otro sistema, nunca en un navegador ni en el código de una página.

PARA TERMINAR

8 Probarlo, lista de control y problemas frecuentes

Probarlo en 5 minutos (código del repositorio)

```
# Python 3.8+, sin instalar nada
set TIZADA_URL=https://tizada.ejemplo.com
set TIZADA_LLAVE=tzp_...
python integracion/codigo/enviar_pedido.py mi_pedido/ --solo-revisar   ← sólo revisa
python integracion/codigo/enviar_pedido.py mi_pedido/                  ← manda, espera y baja los PDF (sha256)
python integracion/codigo/recibir_aviso.py                             ← recibe el aviso en :8090/aviso

# Node 18+
node integracion/codigo/cliente.mjs revisar pedido.json
node integracion/codigo/cliente.mjs mandar OV-2026-00123.zip
node integracion/codigo/cliente.mjs estado OV-2026-00123
node integracion/codigo/cliente.mjs bajar OV-2026-00123
```

O el **sistema de ventas de prueba** del mismo repositorio (`py app.py` con `TIZADA_URL` y `TIZADA_LLAVE`): arma pedidos como en TIZADA, los manda, muestra el avance, recibe el aviso y abre los PDF. Es la referencia funcionando.

Lista de control

- ☐ TIZADA actualizado, Node instalado, el robot «Prendido».
- ☐ Llave creada y guardada en el servidor del otro sistema.
- ☐ El artículo del otro sistema guarda la **clave de la variable** (`v_…`).
- ☐ Telas por **id**, piezas por nombre **genérico**, talles **tal cual**.
- ☐ `referencia` única; `pedido_externo` con lo que identifica la venta.
- ☐ `/pedidos/validar` antes de `/pedidos` .
- ☐ Del resultado se guarda sha256, descarga, drive_id y enlace.
- ☐ El aviso se cree sólo con la firma correcta.

Problemas frecuentes

401: falta `X-Api-Key` o la llave fue anulada.
Queda en «en_cola»: el robot no corre (Integraciones › El robot · `logs/robot.log` · ¿Node instalado?).
«drive-sin-configurar»: falta conectar Drive; los PDF igual se bajan con `descarga` (durante 30 días).
Drive dejó de subir a la semana: la app de Google sigue en «Prueba» (sección 1).
No llega el aviso: TIZADA no llega a esa dirección (red/HTTPS), o la firma no coincide (otra llave).
«variable-ambigua»: mandar la clave `v_…` , no el nombre.
Silueta 404 «sin_calcular»: nadie abrió esa prenda en TIZADA todavía; mostrar un ícono.

Documento generado desde el código del sistema (1.0.48) y probado contra un TIZADA PRO real el 2 de octubre de 2026: un pedido de ejemplo hizo el recorrido completo (revisar, mandar, esperar, bajar 3 tizadas y la ficha con su sha256) y la firma del aviso se comprobó en Python y en Node. Reemplaza las secciones 8 a 13 de «La base de datos y cómo conectar otro sistema» (24/09/2026), que eran una propuesta anterior.